

NMAM INSTITUTE OF TECHNOLOGY, NITTE

Department of Information Science and Engineering

OPERATING SYSTEMS PROGRAMMING ASSIGNMENT REPORT

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem
(Semaphore-based and Monitor-based implementations in Python)

SUBMITTED BY

Name	Disha Kumari
Registration / USN	NNM24IS074
Course	Operating Systems
GitHub Repository	https://github.com/Disha-kumari/OS-Synchronization-Assignment

SUBMITTED TO

Dr Jason Elroy Martis
Associate Professor
Dept Information Science and Engineering, Nitte

1. Problem Statement and Objective

If several threads share common data/resources, then unrestricted access will lead to race conditions and erroneous results. This task realizes two classical synchronization problems via two synchronization mechanisms: semaphores and monitors.

The problems are:

1. Readers-Writers with semaphores
2. Readers-Writers with a monitor
3. Dining Philosophers for four philosophers with semaphores
4. Dining Philosophers for four philosophers with a monitor

The goal is to show mutual exclusion, synchronization, deadlock prevention/avoidance, and starvation issues. Also, all programs record their events during the execution and produce an HTML-based simulation according to the real execution trace.

It is difficult to examine such a program as the scheduling algorithm, not the programmer, decides the order in which the threads are executed. Therefore, the program behavior may vary across executions. There are two classical ways of controlling concurrent programs: semaphores introduced by Dijkstra [6] and monitors introduced by Hoare [8]. Two classical problems that are analyzed using these tools are Readers-Writers problem [7] and Dining Philosophers problem suggested by Dijkstra. All four programs in this project are written in Python language using its standard threading library [1].

In addition to that, the assignment covers the study of synchronization strategies used to control the threads' access when multiple threads run simultaneously. In contrast to just testing the output, the programs log the essential events of their execution process to be able to see what happens to each thread – whether it is waiting for some event, entering critical sections or releasing a resource.

The HTML simulations demonstrate these events. This enables to draw parallels between the theoretical notions of operating system synchronization and the actual execution of Python programs. The same approach is utilized for both synchronization problems to compare the two types of solution.

2. Readers-Writers - Semaphore Solution

Shared Variable: An integer called "value". Readers will read the variable, while writers will increase it.

Semaphores: `mutex=1` will protect the number of readers, while `rw_lock=1` will be for exclusive access to the shared variable [1].

First reader will acquire `rw_lock`; this will prevent writers from accessing it. Last reader will release `rw_lock`.

Writer will acquire `rw_lock` exclusively before modifying the shared variable.

Critical Section: Access to the shared variable. Several readers can access it simultaneously, but a writer should do it separately.

This particular implementation utilizes a readers-preference model, thus writers can theoretically wait for an indefinite period of time should more readers keep on arriving[2].

This is a classic version of the Readers-Writers problem solution presented by Courtois et al. [7] and based on the semaphore construction introduced by Dijkstra [6]. The update of the shared integer variable is purposefully split into two operations – read the variable value, wait and write `value+1`. Should there be any two writers concurrently present in the critical section, the very brief pause between reading and writing would make the lost update inevitable; therefore, the check of the variable value (equals to the number of writers multiplied by rounds) at the end of the program serves as a basic mutual exclusion test. Reader and writer events are registered within the critical section in the same order as the threads actually acquire it.

The primary purpose of this particular solution is to let several readers access the shared resource simultaneously while allowing the writer to have exclusive access. Mutex semaphore protects the reader count variable from being updated by two readers at once. The `rw_lock` provides access control for the shared resource itself.

Once the first reader enters, the resource is locked for the readers collectively. Subsequently, other readers will be able to read from the resource without having to lock the resource exclusively for themselves. Once the last reader exits, the resource will unlock itself so that the waiting writer can enter. This is an example of synchronization being used for concurrency in safe situations.

Information gathered through the execution of the program will serve as further evidence of this situation. The resulting simulation will demonstrate the ability of readers to enter the resource together and the writers waiting for the readers to finish their part.

3. Readers-Writers - Monitor Solution

A class called RWMonitor implements the monitor via a lock and a condition variable.

The monitor keeps track of the number of active readers, whether there is an active writer, and the number of waiting writers.

A reader will block if there is an active writer or if there are any waiting writers.

A writer will block if there are any active readers or if there is an active writer.

notify_all() method is called to unblock threads that are waiting on a shared resource.

Writer preference mechanism prioritizes waiting writers over new readers, making starvation unlikely.

Since in a monitor all methods are run under a single lock, only one thread can modify the shared variables at a time [8].

When a thread cannot proceed, it calls the wait() method, releasing the lock and putting itself to sleep. Once it wakes up, it evaluates its condition within the while loop since another thread might have altered the state during the period of blocking. This is done by using the while loop instead of if, which is a general principle in monitors.

The monitor approach creates an organized manner of managing the shared resource. Instead of managing several synchronization operations performed in different sections of the code, the state to be synchronized and the synchronization operation are encapsulated within the RWMonitor class. This helps in the organization of the criteria of entry and exit from the critical section.

The criteria used to decide which thread will enter next is the number of active readers, writer state, and number of waiting writers. In case of active writer, readers have to wait and vice versa. The condition variable is helpful in letting the waiting threads go to sleep rather than keep checking the resource.

This is because of the fact that a continuous stream of readers coming should not stop the waiting writer from accessing the resource forever once the resource becomes free. Execution trace can be seen for both the waiting and waking up behavior of the monitor approach.

4. Dining Philosophers (4) - Semaphore Solution

There are precisely four philosophers and four forks involved in the implementation.

Each fork has its own semaphore that is set to 1.

Philosopher i uses fork i and fork $(i+1) \% 4$.

Another semaphore named room is set to 3. Consequently, there can be no more than three philosophers trying to take forks at any particular moment of time.

It eliminates the circular-wait problem when all four philosophers have one fork each and need the second fork.

Thus, this implementation avoids the possibility of deadlock.

Starvation cannot be totally eliminated due to the lack of FIFO ordering in the Python semaphore.

Fork i is located between philosopher i and philosopher $(i+1) \% 4$, so each fork belongs to two neighbors. Philosophers continuously perform the same actions: thinking, becoming hungry, taking forks, eating and returning forks. Ownership of the fork is recorded right after a fork was taken and right before it was returned, thus, the HTML output displays the information about who owns what fork.

The four philosophers continuously go through phases such as thinking, becoming hungry, getting the necessary forks, eating, and lastly giving back the forks. Each fork can only be used by a single philosopher at a time as it is a semaphore that has been initialized to 1.

The extra room semaphore is crucial in this case. It ensures that not all the four philosophers compete at the same time for the fork. As a result, there won't be a situation where all the four philosophers will have one fork and be waiting for the second fork forever.

The HTML recording has made the process of resource allocation simpler. Once a philosopher acquires the fork, an event is recorded and once he gives it out, an event is generated. Hence, the simulation allows the allocation of four forks in the real execution of the program.

5. Dining Philosophers (4) - Monitor Solution

The Monitor class in this case is Table.

Table keeps track of the owner of each of the four forks.

It is possible to acquire both forks only if they are both available.

Both forks are acquired after checking them using the monitor lock.

A philosopher does not have one fork in hand while he is trying to acquire the other, hence no hold-and-wait condition exists.

This implementation of the monitor solves the problem of deadlock.

There is a possibility of starvation in this case if neighboring philosophers keep getting the chance to eat while the other philosopher is still waiting.

Each of the four forks has an owner entry in the monitor.

A hungry philosopher waits on the condition variable until both neighboring forks are available. After eating, he releases both forks and calls `notify_all()`, which makes the waiting philosophers check their condition. Since both checks and pickup are done in one go, both neighbors cannot claim that a particular fork is available at the same time.

The monitor solution differs from the semaphore implementation regarding fork acquisition. Instead of letting a philosopher obtain the forks separately, the monitor determines whether both forks are available before allocating them to the philosopher. Resource allocation is, thus, done in one step and is coordinated.

In case a philosopher is hungry, he will wait in case any of his two forks is not available. Upon becoming available, both forks are allocated to the philosopher. Once the philosopher eats, he releases both forks and wakes up the philosophers who were waiting.

Thus, there won't be a case where a philosopher holds one fork and is waiting for the other one. Deadlock condition of hold-and-wait is, thus, prevented. Information regarding who owns which fork can be provided through the monitor and thus allows the HTML simulation to show fork owners.

6. Event Recording and HTML Generation

The synchronization programs keep track of state change events through real execution.

Each thread will invoke the methods such as `rec.state(...)` and `rec.fork(...)` whenever there is a change in their state or resources owned. The recorder captures these events along with time information.

Once all the threads finish their execution, the method `write_html()` creates the HTML page out of these captured events. The events are embedded within the HTML page and JavaScript plays back the captured execution.

In case of Readers-Writers, the animation shows the reader and writer states, shared resource activities and events.

In case of Dining Philosophers, the animation shows all the four philosophers, four forks, philosopher states, and fork ownerships.

Thus, the HTML animations are generated from actual execution of the programs rather than being created separately.

The recorder uses its own little lock for protecting its list of events, thus preventing events from different threads from overwriting each other; and every event includes a time stamp measured since the beginning of the run. It is these time stamps that appear in the event log of every simulation.

The recording of events is helpful due to the fact that the concurrent execution of the program does not always go in the same order, and different threads may have different scheduling for different executions. Thus, recording of the events allows the generation of HTML code that represents the execution of the particular program rather than some manually created fixed order.

The recording of events allows to identify when readers and writers are waiting, reading or writing in Readers-Writers programs; and when philosophers become hungry, get forks, eat and free forks in the Dining Philosophers programs.

Thus, the output of HTML code serves as a replay of the execution of the program.

7. Execution Traces

7.1 Readers-Writers Using Semaphore

The illustration depicts readers going into the reading portion while the writers gain exclusive access to the common resource.

Example sequence:

0.018s R2 -> waiting

0.018s R2 -> reading (value=0)

0.022s R1 -> waiting

0.022s R1 -> reading (value=0)

0.076s W2 -> waiting

0.118s R2 -> idle

0.122s R1 -> idle

0.122s W2 -> writing (value 0->1)

The exact order varies between executions because the programs use concurrent threads and random delays.

7.2 Readers-Writers Using Monitor

The execution shows readers and writers entering the monitor and waiting when the required condition is not satisfied.

Example sequence:

0.031s W1 -> waiting

0.031s W1 -> writing (value 0->1)

0.097s R1 -> waiting

0.114s R2 -> waiting

0.132s W1 -> idle

0.132s R1 -> reading (value=1)

0.132s R2 -> reading (value=1)

The monitor checks its conditions again after a waiting thread is awakened.

7.3 Dining Philosophers Using Semaphore

The execution shows philosophers moving through different states and acquiring/releasing forks.

Example sequence:

0.059s P2 -> hungry

0.059s P2 -> acquiring

0.059s Fork F2 taken by P2

0.065s Fork F3 taken by P2

0.065s P2 -> eating

0.078s P1 -> hungry

0.078s P1 -> acquiring

0.078s Fork F1 taken by P1

The room = 3 semaphore prevents all four philosophers from simultaneously entering the fork-acquisition stage.

7.4 Dining Philosophers Using Monitor

The execution shows philosophers waiting until both required forks are available.

Example sequence:

0.025s P3 -> hungry

0.026s P3 -> acquiring

0.026s Fork F3 taken by P3

0.026s Fork F0 taken by P3

0.026s P3 -> eating

0.126s P3 -> releasing

0.126s Fork F3 released

The monitor acquires both forks atomically, so a philosopher does not keep one fork while waiting for the other.

8. Race Conditions, Mutual Exclusion, Deadlock and Starvation

Race Condition

A race condition is possible when multiple threads read and write to shared variables without synchronization. If, for instance, there are two writers who read the same data before anyone of them writes to it, an update may be lost.

The readers-writers implementations use synchronization techniques to avoid simultaneous conflicting access to the shared variable.

Mutual Exclusion

Mutual exclusion guarantees that only one writer accesses the shared variable. This is done through `rw_lock` in semaphore implementation and through monitor lock and conditions in monitor implementation.

Dining Philosophers: Each fork may belong only to one philosopher.

This can be avoided by using only three philosophers to compete for their forks.

The monitor implementation avoids this situation by acquiring two forks only if they are available.

For deadlock to occur all four conditions have to be present simultaneously: mutual exclusion, hold-and-wait, no preemption and circular wait [9]. This is because in semaphore implementation it breaks the circular wait condition since `room=3` and maximum of three philosophers can have forks and at least one of them can acquire both. In monitor implementation it breaks hold and wait condition since philosopher does not hold one fork while waiting for another.

Starvation

Starvation refers to a situation where a thread/philosopher is waiting for long due to some other threads being served repeatedly.

In semaphore implementation of Readers-Writers problem it follows reader preference approach; hence writers may get starved if readers keep coming [4].

In monitor implementation of Readers-Writers problem it follows writer preference approach hence starving writers gets priority over arriving readers.

In Dining Philosophers problem starvation is possible since there is no strict FIFO guarantee of fairness though deadlock can be avoided.

9. Semaphore vs Monitor Comparison

Aspect	Semaphore	Monitor
Structure	Uses synchronization primitives such as semaphores	Combines shared data, a lock, and condition variables
Waiting	Threads block using acquire()	Threads wait using condition variables
Resource control	Programmer explicitly performs acquire() and release() operations	Resource access is controlled inside monitor methods
Error risk	Incorrect acquire()/release() ordering can cause synchronization problems	Encapsulation makes synchronization easier to organize
Level of abstraction	Lower-level synchronization mechanism	Higher-level synchronization abstraction
Conditions	Conditions are usually managed explicitly by the programmer	Conditions are represented using condition variables

Conclusion: Semaphores have some flexibility but leave the responsibility for correctness of each acquire and release on the programmer, while the monitor places all three in one package, making it easier to verify that the rules are met [8][10].

10. Screenshots of Generated HTML Simulations

The following images shows outputs pf HTML stimulation:

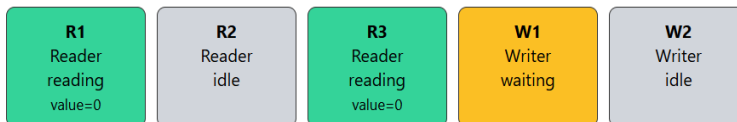
10.1 Readers-Writers Using Semaphore

Readers-Writers (Semaphore)

Semaphores: mutex=1, rw_lock=1 | readers-preference

▶ Start || Pause ⏮ Reset Speed 1 5 / 45

Shared resource: **READ by 2 reader(s)**



Event log (from program trace)

```
0.048s R1 -> waiting
0.048s R1 -> reading (value=0)
0.113s R3 -> waiting
0.113s R3 -> reading (value=0)
0.128s W1 -> waiting
```

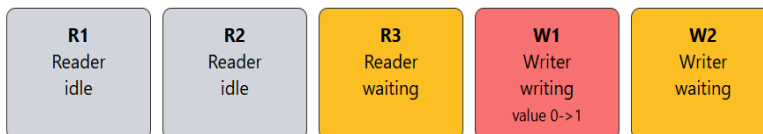
10.1 Readers-Writers Using Monitors

Readers-Writers (Monitor)

Monitor: Condition(lock) | writer-preference

▶ Start || Pause ⏮ Reset Speed 1 4 / 45

Shared resource: **WRITING (exclusive)**



Event log (from program trace)

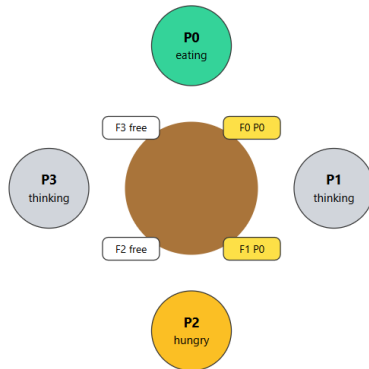
```
0.032s W1 -> waiting
0.032s W1 -> writing (value 0->1)
0.065s W2 -> waiting
0.097s R3 -> waiting
```

10.3 Dining Philosophers Using Semaphore

Dining Philosophers x4 (Semaphore)

Semaphores: fork[0..3]=1 each, room=3 (deadlock avoidance)

▶ Start || Pause ⏮ Reset Speed 1 10 / 148



Event log (from program trace)

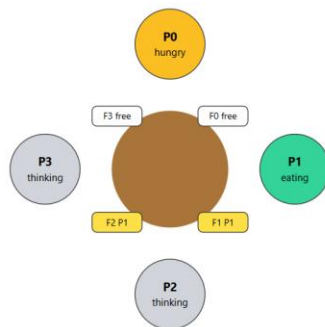
```
0.000s P0 -> thinking
0.000s P1 -> thinking
0.000s P2 -> thinking
0.000s P3 -> thinking
0.033s P0 -> hungry
0.033s P0 -> acquiring
0.033s Fork F0 taken by P0
0.027s Fork F1 taken by P0
```

10.4 Dining Philosophers Using Monitor

Dining Philosophers x4 (Monitor)

Monitor: Condition(lock), both forks taken atomically

▶ Start || Pause ⏮ Reset Speed 1 6 / 144



Event log (from program trace)

```
0.046s P1 -> hungry
0.046s P1 -> acquiring
0.046s Fork F1 taken by P1
0.046s Fork F2 taken by P1
0.046s P1 -> eating
0.094s P0 -> hungry
```

11. AI Usage Summary

Claude (Anthropic) was employed during the development process for planning, programming, debugging, explaining and HTML visualization [5].

All key prompts and corresponding prompts used in the development process are mentioned in `ai_prompts/prompts.md`.

The programs were tested and the synchronization behavior verified by running the code. The final explanations were analyzed for understanding the role of semaphores, monitors, locks, condition variables, critical sections, deadlock prevention and starvation.

An important limitation discovered during the development process is the fact that Python semaphores and condition variables do not guarantee a strict FIFO property. Thus starvation is always possible even when deadlock is prevented.

12. References

- [1] Python Software Foundation, "threading — Thread-based parallelism," Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/threading.html>
- [2] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts.
- [3] A. S. Tanenbaum and H. Bos, Modern Operating Systems.
- [4] A. B. Downey, The Little Book of Semaphores. [Online]. Available: <https://greenteapress.com/wp/semaphores/>
- [5] Anthropic, "Claude," AI assistant.
- [6] E. W. Dijkstra, "Cooperating sequential processes," Technological University Eindhoven, EWD123, 1965.
- [7] P. J. Courtois, F. Heymans, and D. L. Parnas, "Concurrent control with 'readers' and 'writers'," Communications of the ACM, vol. 14, no. 10, pp. 667–668, 1971.
- [8] C. A. R. Hoare, "Monitors: An operating system structuring concept," Communications of the ACM, vol. 17, no. 10, pp. 549–557, 1974.
- [9] E. G. Coffman, M. J. Elphick, and A. Shoshani, "System deadlocks," ACM Computing Surveys, vol. 3, no. 2, pp. 67–78, 1971.
- [10] W. Stallings, Operating Systems: Internals and Design Principles, Pearson.

13. Academic Integrity Declaration

I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own.

Name: Disha Kumari

USN: NNM24IS074

Signature: _____

Date: _____